Desenvolvimento para Web

Padrões de arquitetura e banco de dados

Prof. João Pedro Winckler Bernardi
Ciência da Computação / Sistemas da Informação
Unochapecó

Revisado em 10 de abril de 2026

Roteiro da Aula

1. Retrospectiva
2. Padrões de Arquitetura
3. MVC
4. Arquitetura em camadas
5. Aplicando arquitetura em camadas/incorporando Banco de Dados
6. Exercícios
7. Revisão

Retrospectiva

Retrospectiva

O que vimos anteriormente

Entendemos como aplicar estilos em páginas HTML usando CSS, como selecionar elementos, alterar cores e tamanhos, entender o espaçamento dos elementos e adaptar o layout para diferentes tamanhos de tela.

E agora?

Hoje, voltaremos ao back-end, visando padrões de arquitetura e banco de dados.

- Compreender a importância da arquitetura em aplicações web
- Entender como organizar melhor o código
- Conhecer padrões arquiteturais comuns
- Preparar a aplicação para uso de banco de dados
- Migrar dados em memória para SQLite

Padrões de Arquitetura

Por que falar de arquitetura?

- Organização do código
- Separação de responsabilidades
- Facilidade de manutenção
- Escalabilidade
- Trabalho em equipe

MVC

Padrão MVC

- **M**odel = dados e regras de negócio
- **V**iew = interface apresentada ao usuário
- **C**ontroller = recebe requisições e coordena o fluxo

O MVC separa a aplicação em responsabilidades distintas, facilitando organização e manutenção.

Responsabilidade de cada camada

- **Model**
  - representa os dados do sistema
  - contém regras de negócio
  - conversa com o banco de dados
- **View**
  - responsável pelo que o usuário visualiza
  - retorna páginas HTML completas, templates ou telas
  - exibe os dados recebidos
- **Controller**
  - recebe a requisição
  - processa a rota
  - decide qual model usar
  - escolhe qual view retornar

Exemplo do fluxo MVC

1. Usuário acessa `/usuarios`
2. Controller recebe a requisição
3. Controller consulta o Model
4. Model busca os dados
5. Controller envia os dados para a View
6. View retorna a página HTML ao navegador

O MVC deixou de ser usado?

Resposta curta

Não. O MVC ainda existe e ainda pode ser encontrado em muitas empresas.

- Muito comum em sistemas legados
- Presente em frameworks clássicos
- Bastante usado em aplicações monolíticas
- Ainda é importante para aprendizado

Por que o MVC perdeu espaço?

- Separação entre frontend e backend
- APIs REST retornando JSON
- Frameworks modernos no frontend
- Aplicações distribuídas e microsserviços

Hoje, muitas aplicações não retornam páginas completas no backend.

Antes e hoje

Antes (MVC clássico):

- Backend retorna HTML completo
- View no servidor

Hoje:

- Backend retorna JSON
- Frontend renderiza a interface
- Backend focado em regras e dados

Arquitetura em camadas

Arquitetura em camadas

- **Controller**: entrada e saída
- **Service**: regras de negócio
- **Repository**: acesso aos dados

Cada camada possui uma responsabilidade bem definida.

Camada Controller

- Recebe a requisição HTTP
- Lê parâmetros da rota
- Recebe dados do body
- Chama o service
- Retorna a resposta

Exemplo: `GET /usuarios`

Camada Service

- Contém a lógica do sistema
- Faz validações
- Aplica regras de negócio
- Decide o fluxo da operação

Exemplo: verificar se o usuário já existe

Camada Repository

- Responsável pelo acesso ao banco
- Executa consultas SQL
- Insere, atualiza e remove dados
- Não contém regra de negócio

Exemplo: `SELECT * FROM usuarios`

Fluxo da requisição

Usuário $\rightarrow$ Controller $\rightarrow$ Service $\rightarrow$ Repository $\rightarrow$ Banco

Retorno:
Banco $\rightarrow$ Repository $\rightarrow$ Service $\rightarrow$ Controller $\rightarrow$ Cliente

Aplicando arquitetura em camadas/incorporando Banco de Dados

Por que usar banco de dados?

- Persistência dos dados
- Dados não são perdidos ao reiniciar
- Melhor organização
- Consultas mais eficientes
- Aproximação com sistemas reais

Problema da memória

Exemplo

```
let usuarios = [];
```

- Funciona apenas enquanto a aplicação está rodando
- Reiniciou o servidor = perdeu tudo
- Não é viável em produção

Instalando o SQLite

Comando

```
npm install sqlite3
```

- Banco leve
- Fácil de configurar
- Ideal para aprendizagem
- Não precisa de servidor separado

Antes: dados em memória

```
let usuarios = [];

usuarios.push({
    id: 1,
    nome: "João"
});
```

- Simples
- Fácil para iniciar
- Dados temporários

O que muda com o banco?

- Não usamos mais arrays
- Não usamos mais `push()`
- Consultas passam a usar SQL
- Dados ficam persistidos

Bootstrap do servidor

Quando o servidor inicia, algumas tarefas precisam acontecer antes de atender requisições:

- iniciar o Express
- configurar middlewares
- conectar ao banco
- criar tabelas necessárias

Esse processo inicial é chamado de **bootstrap**.

Por que criar a tabela no bootstrap?

- garantir que a estrutura exista
- evitar erros nas primeiras consultas
- automatizar a configuração do sistema
- facilitar testes e desenvolvimento

Sem a tabela, o CRUD não funciona.

Onde colocamos a conexão?

Sugestão de estrutura do projeto:

- `app.js` → servidor e rotas
- `database.js` → conexão e bootstrap do banco

Separar responsabilidades melhora a organização.

Arquivo database.js

```
const sqlite3 = require('sqlite3').verbose();

const db = new sqlite3.Database('./database.db');

module.exports = db;
```

Arquivo `app.js`

```
const express = require('express');
const db = require('./database');

const app = express();
```

Bootstrap da tabela no `app.js`

```
db.run(`
    CREATE TABLE IF NOT EXISTS mensagem (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        texto TEXT NOT NULL
    )
`);

app.listen(3000);
```

A criação da tabela ocorre antes do servidor iniciar.

Próxima etapa: migrar mensagens

Agora vamos migrar a funcionalidade de mensagens para a nova arquitetura:

- criar a tabela `mensagem`
- criar o primeiro `Repository`
- criar o primeiro `Service`
- preparar a rota para usar as camadas

Passo 1: tabela mensagem

```
db.run(`
    CREATE TABLE IF NOT EXISTS mensagem (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        texto TEXT NOT NULL
    )
`);
```

Estrutura das camadas

- `repository/MensagemRepository.js`
- `service/MensagemService.js`
- `controller/MensagemController.js`

Separaremos acesso a dados e regras de negócio.

Passo 2: primeiro Repository

```
const db = require('../database');

function listAll(callback) {
    db.all(
        'SELECT * FROM mensagem',
        [],
        callback
    );
}

module.exports = { listAll };
```

Responsabilidade do Repository

- executar SQL
- buscar dados
- inserir registros
- atualizar e remover

Não colocamos regra de negócio aqui.

Passo 3: primeiro Service

```
const mensagemRepository = require('../repository/MensagemRepository');

function listAll(callback) {
    mensagemRepository.listAll(callback);
}

module.exports = { listAll };
```

Passo 4: primeiro Controller

```
const mensagemService = require('../service/
MensagemService');

function listAll(req, res) {
    mensagemService.listAll((err, dados) => {
        res.json(dados);
    });
}

module.exports = { listAll };
```

Problema atual

Se colocarmos todas as rotas no `app.js`, o arquivo cresce rapidamente.

- difícil de manter
- difícil de localizar rotas
- mistura responsabilidades

Solução: arquivo de rotas

Nova estrutura:

- `app.js`
- `routes/mensagemRoutes.js`
- `controller/`

Cada conjunto de rotas fica em seu próprio arquivo.

Arquivo mensagemRoutes.js

```
const express = require('express');
const router = express.Router();

const controller = require('../controller/
MensagemController');

router.get('/mensagem',
    controller.listAll);

module.exports = router;
```

Importando no app.js

```
const mensagemRoutes =
    require('./routes/mensagemRoutes');

app.use(mensagemRoutes);
```

Exercícios

Exercício: cadastrar uma nova mensagem

Objetivo

Implemente a funcionalidade de cadastro de mensagens. Para isso, você deve criar a rota `POST /mensagem` para inserir uma nova mensagem no banco de dados.

Passos sugeridos:

1. criar o método no `Repository`
2. criar o método no `Service`
3. criar o método no `Controller`
4. adicionar a rota no arquivo de rotas
5. testar com Insomnia ou Postman

O que precisa existir

Sua implementação deve conter:

- `insert` no Repository usando SQL
- chamada no Service
- resposta HTTP no Controller
- rota `POST`

A mensagem será recebida pelo `req.body`.

Próximo exercício: cadastro de usuários

Agora implemente o cadastro de usuários no banco.

Objetivo: Criar a rota `POST /usuarios`

A tabela deve conter, no mínimo:

- `id`
- `nome`
- `email`
- `senha`

Atenção: A senha **não pode** ser salva em texto puro.

Protegendo a senha

Instalação:

```
npm install bcrypt
```

Uso no Service:

```
const bcrypt = require('bcrypt');

const senhaHash =
    await bcrypt.hash(senha, 10);
```

Revisão

Revisão da Aula

O que aprendemos hoje

- A importância da **arquitetura** em aplicações web.
- O funcionamento do padrão **MVC**.
- Por que o MVC clássico perdeu espaço em sistemas modernos.
- A arquitetura em camadas: **Controller – Service – Repository**.
- Como configurar o **SQLite** no servidor.
- Como criar tabelas durante o **bootstrap**.
- Como migrar dados da memória para o banco.
- Como organizar as **rotas** em arquivos separados.
- Como implementar o fluxo completo: rota → controller → service → repository → banco.
- A importância de armazenar senhas com **hash usando bcrypt**.